WatchaMe Operations Guide

Operating a self-updating Kodi add-on repository, and provisioning a Fire TV Stick 4K from it.

Two procedures and a worksheet. **Book One** sets up GitHub and runs the repository — the thing that keeps add-ons updating by themselves. **Book Two** takes a Fire TV Stick from factory state to a finished box in 34 numbered steps. **Book Three** is a worksheet to fill in as you go.

Start at Book One, Part 1 if GitHub has never been set up on this machine. If the repository is already published, you only need Book Two.

Where the work happens

Book One is terminal-driven with three unavoidable browser moments. Book Two is done on the TV with a remote, with a few checks from your computer.

	Terminal	Browser	On the TV
Book One 1 GitHub setup	install, git config, `gh auth login`	create the account; approve the sign-in	—
Book One 2 Configure	build, check, commit	—	—
Book One 3 Choose add-ons	all of it	—	—
Book One 4 Publish	create repo, push, verify URLs	4.5 workflow permission	—
Book One 5–7 Remove/change	all of it	—	—
Book One 8 Daily operation	nothing — it runs itself	—	—
Book One 9 Health check	all of it	—	9.6
Book Two A–F Provisioning	optional, but faster for step 17	—	all of it
Book Two G Verify	steps 31–33	—	step 34

Every part carries a **Where** line repeating this, so you never have to guess which screen you should be looking at. **If you would rather not use a terminal at all,** Book One, Appendix D walks the same work through GitHub's web interface — the build runs on GitHub either way, so nothing is lost but local testing.

Each step is one command to type or one control to click, and lines marked **Check** tell you what success looks like — if you do not see it, stop there rather than continuing. Background, file maps and

troubleshooting live in the appendices so the procedures stay short.

Three files in the repository root pair with these instructions: `addons.wishlist.txt` is where you list the add-ons you want (Part 3 turns it into real entries for you), `addons.template.json` holds the entry shapes if you would rather write them by hand, and `repo.config.json` is the identity you set in Part 2.

Contents

BOOK ONE

GitHub setup and running the repository

Every section here is a numbered procedure. Run the steps in order; each one is a command to type or a control to click. Lines marked **Check** tell you what success looks like — if you do not see it, stop there rather than continuing.

Start at Part 1 if you have never set up GitHub on this machine. If `gh auth status` already reports you as logged in, skip to Part 2. **Prefer not to use a terminal at all?** Appendix D is the same work done entirely in a web browser.

First time through, read Parts 1 to 4 in order and stop there:

```
1  GitHub setup      account, tools, sign in
2  Configure         name the repository
3  Choose add-ons    decide what it carries      <- do this BEFORE publishing
4  Publish           push it live, verify the URLs
   then Book Two     install on the device       <- last, and only once the above is d
one
```

Parts 5 to 9 are for afterwards: removing an add-on, changing the Toolbox, moving the repository, and checking its health. You do not need them on a first run.

Background and troubleshooting live in the appendices, so the procedures stay short.

Part 1 — GitHub setup

Where: terminal on your computer, plus a browser for 1.1 and the sign-in approval in 1.4. Skip this part entirely if you are using the browser-only route in Appendix D.

Do this once per machine.

1.1 Create a GitHub account — skip if you have one.

Go to **github.com/signup** and register. Your username becomes part of every URL your devices fetch from, so pick one you are willing to keep.

1.2 Install the tools.

macOS (Homebrew):

```
brew install git gh
```

Debian / Ubuntu:

```
sudo apt update && sudo apt install git gh python3
```

Windows:

```
winget install Git.Git GitHub.cli Python.Python.3.12
```

Check: all three report a version.

```
python3 --version     # need 3.8 or newer
```

```
git --version
gh --version
```

1.3 Tell git who you are. Commits fail or land under the wrong name without this.

```
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
```

Check:

```
git config --global user.name && git config --global user.email
```

> Use the email attached to your GitHub account, or commits will not be linked to your profile. GitHub also offers a `@users.noreply.github.com` address if you would rather not publish a real one.

1.4 Sign in to GitHub from the terminal.

```
gh auth login
```

It asks four questions. The answers:

Prompt	Answer
What account do you want to log into?	**GitHub.com**
What is your preferred protocol for Git operations?	**HTTPS**
Authenticate Git with your GitHub credentials?	**Yes**
How would you like to authenticate?	**Login with a web browser**

It then shows a one-time code. Copy it, press Enter, and your browser opens; paste the code and approve. The token is stored in your system credential store.

Check:

```
gh auth status
```

You want `Logged in to github.com account YOURNAME` and a token with the `repo`, `read:org` and `gist` scopes.

> **Headless machine, or no browser?** Create a personal access token (classic) at **github.com/settings/tokens** with the `repo`, `read:org` and `gist` scopes, then pipe it in: `gh auth login --with-token < token.txt`.

1.5 Get the repository onto this machine.

If you already have the directory, just note where it is. Otherwise clone it:

```
git clone https://github.com/YOURNAME/YOURREPO.git ~/Dev/kodi-repo
```

Starting from scratch instead? Copy the repository files into a new directory and:

```
cd ~/Dev/kodi-repo && git init -b main
```

Part 2 — Configure the repository

Where: terminal and a text editor. No browser.

2.1 Open the repository directory.

```
cd ~/Dev/kodi-repo
```

2.2 Set your identity. Open `repo.config.json` and set these four fields. Record them in the worksheet (Book Three) as you go — the install URL in Part 3 is built from them.

Field	Set it to
`github_user`	Your GitHub username or org, exactly as it appears in URLs — the **Owner** field on GitHub's create page
`github_repo`	The repository name, matching the **Repository name** field exactly, capitals included
`repo_name`	The display name shown in Kodi. Anything readable.
`provider`	Your name, shown as the author

Leave `repo_id`, `branch`, `hosting` and `keep_versions` alone unless Part 7 applies.

2.3 Build.

```
python3 scripts/build_repo.py
```

Check: the last two lines report the add-on count, an md5, and the URL it will serve from. That URL must contain your `github_user` and `github_repo` from 2.2.

2.4 Check dependencies.

```
python3 scripts/check_deps.py
```

Check: `all dependencies resolve`. If not, go to Appendix C.

2.5 Commit.

```
git add -A && git commit -m "Configure repository identity"
```

Part 3 — Choose your add-ons

Where: terminal and a text editor. No browser.

This is where you decide what your repository carries. Do it before you publish, and again whenever you want to change the set.

You fill in one file and run one command. The whole part is:

```
edit addons.wishlist.txt          list what you want
python3 scripts/plan_addons.py    check it
...fix anything it flags
python3 scripts/plan_addons.py --apply
...write a "why" for each new entry
build, check, push
```

3.1 List what you want. Open `addons.wishlist.txt` in the repository root. One add-on per line:

```
script.trakt
service.subtitles.a4ksubtitles   a4k-openproject/a4kSubtitles
script.plexmod                                      optional
```

Field	Required	Meaning
add-on id	**Yes**	The real id, not the display name. Find it in the add-on's own `addon.xml`, or as the folder name under `~/.kodi/addons/`.
source	Only sometimes	Where it comes from. Needed for anything **not** in Kodi's official repository — you do not have to know which those are; step 3.2 tells you. One of the three forms below.
`optional`	No	Leaves the entry **unticked** in the Toolbox picker. Use it for anything needing credentials, a subscription, or extra hardware.

Everything after a `#` is a comment, so the instructions already in the file can stay.

The source is one of three forms, and you will need one for every add-on that Kodi does not ship itself:

Form	Use when	Example
`owner/repo`	It lives on GitHub	`a4k-openproject/a4kSubtitles`
`https://…/zips`	It is served by another Kodi repository — the directory holding that repo's `addons.xml`	`https://raw.githubusercontent.com/owner/repo/master/omega/zips`
`https://…/x-1.2.3.zip`	All you have is a fixed zip	`https://host/plugin.video.x-1.2.3.zip`

Prefer the first two. Both track the newest version automatically; a fixed zip only changes when that URL's contents change.

You know the repository but not the add-on ids? Point the planner at it and it will list them, already formatted for this file:

```
python3 scripts/plan_addons.py --discover https://that-repo/...
```

Give it the repository's base URL or its repository zip URL. It follows nested indexes — many vendor repos split add-ons across per-Kodi-version directories — and prints every add-on served, newest version first. Copy the lines you want.

3.2 Check the list.

```
python3 scripts/plan_addons.py
```

It reads your wishlist, checks every id against Kodi's official Omega and Piers indexes, and prints one line each:

```
--  service.subtitles.a4ksubtitles    already in addons.json
ok  script.trakt                      official  (installed by id)
ok  plugin.video.example              mirror    (github owner/repo)
ok  script.fromvendor                 mirror    (kodi repo)
ok  script.pinned                     mirror    (fixed zip)
!!  script.something                  NOT in the official repo, and no source given
```

Marker	Meaning	Do
`--`	Already carried	Nothing
`ok … official`	Kodi ships it; it will be installed by id and stay on its maintainer's update channel	Nothing
`ok … mirror`	Not in Kodi's repo; it will be packaged into yours from the source you gave — the bracket says which	Nothing
`!!`	Not in Kodi's repo, and you gave no source	Step 3.3

3.3 Resolve anything marked `!!`. The add-on has to come from somewhere. Add one of the three source forms to that line:

```
script.something   owner/repo
script.something   https://that-repo/omega/zips
script.something   https://host/script.something-1.2.3.zip
```

If it is on GitHub, searching for the exact add-on id usually finds it — most Kodi add-ons use their id as the repository name. If it comes from a vendor's Kodi repository, run `--discover` against that repository and copy the line it prints. Re-run 3.2 until no `!!` lines remain.

If the add-on has no GitHub repository at all and installs only from a vendor's own Kodi repository, it belongs in the `external` section instead. Add it by hand using the `external` shape at the end of this part, and it is documented rather than installed.

3.4 Apply.

```
python3 scripts/plan_addons.py --apply
```

This changes two files:

- `addons.json` — your entries are appended to the right sections.
- `src/script.watchame.toolbox/addon.xml` — the Toolbox's version is bumped for you.

> The version bump is not cosmetic. The curated list ships *inside* the Toolbox, so without it the published index is byte-identical and **no device ever fetches your new list**, while every build reports success. The planner does it so you cannot forget; `build_repo.py` refuses to finish if it is ever missed.

3.5 Write a `why` for each new entry. The planner leaves a placeholder:

```
"why": "TODO: one sentence on why this earned a slot."
```

Open `addons.json` and replace every `TODO`. This text ends up in the README and is the only record of why an add-on is there. It also guesses `name` and `category` from the id — correct those at the same time.

3.6 Build and check.

```
python3 scripts/build_repo.py && python3 scripts/check_deps.py
```

Check: the build ends with an add-on count and an md5, and the dependency check says `all dependencies resolve`.

If it reports an unresolvable import, that add-on needs a dependency nobody publishes — move it to the `external` section (see the end of this part) rather than shipping something that cannot install.

3.7 Commit and push.

```
git add -A && git commit -m "Add <what you added>" && git push
```

Skip the push if you have not done Part 4 yet; just commit and carry on.

3.8 Confirm it landed.

```
grep -o 'id="[^"]*" name="[^"]*" version="[^"]*"' docs/addons.xml
```

Check: every add-on you added appears with a version. On a device, Toolbox → *Install curated add-ons* shows the new entries, unticked if you marked them `optional`.

Writing entries by hand

The planner exists so you do not have to, but the entries are plain JSON and you can add them directly. `addons.template.json` in the repository root holds all three shapes ready to paste.

`addons.json` has three sections. An entry goes in exactly one.

`mirror` — packaged into your repo from GitHub:

```
        {
          "id": "service.vpn.manager",
          "name": "VPN Manager for OpenVPN",
          "category": "privacy",
          "why": "One sentence on what it does and why it earned a slot.",
          "github": "Zomboided/service.vpn.manager",
          "track": "release",
          "optional": true
        }
```

`official` — installed by id from Kodi's own repo. No `github`, no `track`:

```
        {
          "id": "script.trakt",
          "name": "Trakt",
          "category": "tracking",
          "why": "Scrobbling, watched-state sync and lists across devices."
        }
```

`external` — documented, never auto-installed. No `id`; a vendor `url` and the add-ons it provides:

```
        {
          "name": "TheMovieDb Helper (6.x)",
          "why": "Hard-requires script.module.pil, which no official repo ships.",
          "url": "https://jurialmunkey.github.io/repository.jurialmunkey/",
          "addons": ["plugin.video.themoviedb.helper", "script.skinvariables"]
        }
```

Field	Sections	Meaning
`id`	mirror, official	Must match the add-on's own `addon.xml` exactly
`name`	all	Display name in the Toolbox picker
`category`	mirror, official	Groups and sorts the picker. In use: `subtitles`, `playback`, `tracking`, `tools`, `interface`, `privacy`.
`why`	all	Your justification. Ends up in the README.
`github`	mirror	`owner/repo`. One of `github`, `repo_url` or `zip_url` is required.
`repo_url`	mirror	Directory holding another Kodi repo's `addons.xml`; the newest version is taken automatically

Field	Sections	Meaning
`zip_url`	mirror	A fixed zip URL. Pinned — only changes when that URL's contents change.
`track`	mirror	With `github`: `release` (newest release, falling back to branch head) or `branch`
`ref`	mirror	Pin a branch, e.g. `"ref": "matrix"`
`optional`	mirror, official	`true` leaves it unticked in the picker

Adding entries by hand means **bumping the Toolbox version yourself** (step 3.4 explains why), then continuing from 3.6.

Part 4 — Publish to GitHub

Where: terminal, except 3.5 which is a browser setting.

Do this once. After it, everything is automatic.

4.1 Point the local repository at GitHub. It must be public — Kodi cannot authenticate. Which route depends on whether the repository exists yet.

Route A — it does not exist yet. One command creates it and wires up the remote:

```
gh repo create YOURNAME/YOURREPO --public --source=. --remote=origin
```

`YOURNAME` is the owner and `YOURREPO` the repository name — the same two values as `github_user` and `github_repo` in 2.2. It adds no README, .gitignore or license, which is what you want: any of them creates an initial commit and turns your first push into a merge.

Route B — you already created it in the browser (Appendix D.1, or github.com/new). Do not run the command above; it will fail. Just add the remote:

```
git remote add origin https://github.com/YOURNAME/YOURREPO.git
```

Check either route:

```
git remote -v
```

Both lines must show your repository's URL.

> `GraphQL: Name already exists on this account (createRepository)` means the repository is already there — you are on Route B. Nothing is broken and nothing was changed; add the remote and carry on to 3.2.
>
> `remote origin already exists` means the remote is set already. Confirm it points where you expect with `git remote -v`, or repoint it: `git remote set-url origin URL`.

4.2 Push.

```
git push -u origin main
```

Check: no authentication prompt. If you get one, redo 1.4.

4.3 Wait for the build.

```
gh run watch
```

Check: the run completes green. If it fails, read the log — it is running the same two commands you ran in 2.3 and 2.4.

> **No run appears at all?** That is normal on the very first push. The workflow filters on changed paths, and GitHub often skips that filter when a branch is created rather than updated. Start one by hand and carry on:
>
> `gh workflow run sync.yml && gh run watch`

4.4 Verify what Kodi will see. Replace `YOURNAME` and `YOURREPO` in the first line, then paste the whole block.

```
BASE=https://raw.githubusercontent.com/YOURNAME/YOURREPO/main/docs
curl -sI  $BASE/addons.xml     | head -1            # expect: HTTP/2 200
curl -s   $BASE/addons.xml.md5                      # expect: 32 hex characters, nothi
ng else
curl -sI  $BASE/repository.watchame.zip | head -1    # expect: HTTP/2 200
```

Check: all three as described. A 404 on the first two means 2.2 does not match where you actually pushed.

4.5 Confirm Actions can write back. *(Browser.)* The sync job commits the regenerated `docs/` to your repository, so it needs write permission. On a repo you own, Actions are enabled automatically — the "enable workflows" prompt only appears on forks — but the write permission is a separate setting that will fail the job if it is wrong.

Open the repository on github.com → **Settings** → **Actions** → **General** → scroll to **Workflow permissions** → select **Read and write permissions** → Save.

Check: the **Sync repository** workflow appears under the Actions tab, and the run from 3.3 shows a commit step that succeeded. A `403` or `permission denied` on the push step means this setting is still read-only.

A green run does not prove this is right. If nothing changed upstream, the job has nothing to commit and never exercises the permission — it goes green either way, and you find out weeks later when the first real update fails. Check the setting directly instead:

```
gh api repos/OWNER/REPO/actions/permissions/workflow --jq
.default_workflow_permissions
```

It must print `write`. To set it without leaving the terminal:

```
gh api -X PUT repos/OWNER/REPO/actions/permissions/workflow -f
default_workflow_permissions=write
```

4.6 Write down the install URL. This is what you type into Kodi in Book Two, step 17:

```
https://raw.githubusercontent.com/YOURNAME/YOURREPO/main/docs
```

No `gh`? Create the repository at **github.com/new** — public, no README, no .gitignore — then: `git remote add origin https://github.com/YOURNAME/YOURREPO.git && git push -u origin main`. When git asks for a password, paste a personal access token; GitHub stopped accepting account passwords for this in 2021.

Part 5 — Remove an add-on

Where: terminal and a text editor.

5.1 Delete its entry from `addons.json`.

5.2 Bump the Toolbox version in `src/script.watchame.toolbox/addon.xml`.

5.3 Build and push:

```
python3 scripts/build_repo.py && python3 scripts/check_deps.py
git add -A && git commit -m "Remove <addon-id>" && git push
```

Check: `grep '<addon id="<addon-id>"' docs/addons.xml` returns nothing.

> Copies already installed on devices keep working but stop receiving updates. A repository cannot retract an add-on — to remove it from a device, uninstall it there.

Part 6 — Change the Toolbox itself

Where: terminal and a text editor.

6.1 Edit the code under `src/script.watchame.toolbox/`.

6.2 Bump the version in its `addon.xml` — patch for a fix, minor for new behaviour.

6.3 Build, check, push:

```
python3 scripts/build_repo.py && python3 scripts/check_deps.py
git add -A && git commit -m "Toolbox: <what changed>" && git push
```

Part 7 — Change the repository's identity or URLs

Where: terminal and a text editor, then each device.

Only when moving the repo or renaming it.

7.1 Edit `repo.config.json`: `github_user`, `github_repo`, `branch`, or `hosting`.

7.2 Bump `repo_version` in the same file.

7.3 Build and push as in 6.3.

7.4 Reinstall the repository zip on **every device.** The old URLs are baked into the copy they already have.

> Because of 7.4, get `github_user` and `github_repo` right in Part 2 and leave them alone.

Part 8 — Daily operation

Where: nowhere — it runs itself.

Nothing to do. For reference, the scheduled job at 05:15 UTC:

1. Checks each mirrored add-on's upstream for a newer release.
2. Repackages anything that moved.
3. Runs the dependency check; a failure stops the commit.
4. Commits the regenerated `docs/` only if something changed.

Devices poll roughly every 24 hours, so worst case is about 48 hours from an upstream release to your stick.

To force a run now:

```
gh workflow run sync.yml && gh run watch
```

Part 9 — Health check

Where: terminal, plus the device for 9.6.

Run after any push, or whenever something looks wrong.

9.1 The index lists what you expect.

```
grep -o 'id="[^"]*" name="[^"]*" version="[^"]*"' docs/addons.xml
```

9.2 A specific add-on is present.

```
grep -o 'id="service.vpn.manager"[^>]*version="[^"]*"' docs/addons.xml
```

9.3 Its zip is Kodi-shaped — `<id>/addon.xml` must be at the root:

```
unzip -l docs/zips/service.vpn.manager/*.zip | head -5
```

9.4 The Toolbox carries the current list.

```
python3 -c "import zipfile,json,glob; z=sorted(glob.glob('docs/zips/script.watchame.to
olbox/*.zip'))[-1]; \
m=json.loads(zipfile.ZipFile(z).read('script.watchame.toolbox/resources/addons.json'))
; \
print(sorted(e['id'] for s in ('mirror','official') for e in m[s]))"
```

9.5 The live URLs still respond. Re-run 4.4.

9.6 On a device: Toolbox → Install curated add-ons. New entries appear in the picker, unticked if you marked them `optional`.

Appendix A — How it works

The update loop. Kodi downloads `addons.xml.md5` (32 bytes) on a ~24 hour cycle. If that hash changed, it downloads `addons.xml`, compares every `version=` against what is installed, and downloads anything newer from `docs/zips/`. So an update happens **only** when a version number increases. Re-uploading a zip at the same version does nothing; lowering a version does nothing. A broken release is fixed with a higher patch version, never a re-upload.

The three routes. `official` keeps an add-on on its maintainer's update channel — mirroring something Kodi already ships would fork it onto your channel and make you responsible for tracking it. `mirror` is for add-ons genuinely unavailable elsewhere. `external` is for add-ons whose dependencies nobody publishes: TheMovieDb Helper 6.x hard-requires `script.module.pil`, a compiled Pillow build no official repo ships, so mirroring it would deliver an add-on that cannot install.

What the builder does. Syncs mirrors from upstream; packages everything in `src/` at the version in its `addon.xml`; generates the repository add-on from `repo.config.json`; regenerates the index, checksum, `index.html`, and prunes old zips. Zips use fixed timestamps, so identical content produces identical bytes

— otherwise CI would commit noise daily.

What the dependency checker does. Fetches Kodi's official index for Omega and Piers, unions it with your own IDs, and verifies every non-optional `<import>` resolves. It fails the build rather than letting a device discover the problem.

Where the add-on ID appears. Folder name in `docs/zips/`, zip filename, folder at the root inside the zip, and the `id=` in `addon.xml`. Kodi builds its download URL from the ID and version in the index, so any mismatch is a 404 that surfaces as "failed to install". The builder derives all four from one source, which is why you never rename zips by hand.

Appendix B — File map

Path	Purpose	Edit?
`addons.json`	The curated list	**Yes — this is the control surface**
`addons.wishlist.txt`	**Fill this in** — what you want the repo to carry	**Yes**
`addons.template.json`	Copy-paste entry templates; never read by the build	Reference
`scripts/plan_addons.py`	Turns the wishlist into `addons.json` entries	Rarely
`repo.config.json`	Repo identity, hosting, versions	Yes
`src/`	Add-ons you maintain	Yes
`scripts/build_repo.py`	The builder	Rarely
`scripts/check_deps.py`	Dependency validation	Rarely
`scripts/make_art.py`	Regenerates icon/fanart (needs Pillow)	Rarely
`assets/`	Committed icon and fanart	Rarely
`docs/`	**Published tree — what Kodi downloads**	**Never by hand**
`guides/`	This document's source	Yes

Appendix C — Troubleshooting

Symptom	Cause	Fix
`gh auth login` opens no browser	Headless machine	Use the token method in the 1.4 note
`git push` asks for a password	Not authenticated, or using a password	Redo 1.4, or paste a personal access token
`Name already exists on this account`	The repository was already created, probably in the browser	Skip to 4.1 Route B
`destination path already exists and is not an empty directory`	Cloning on top of the local repository you already have	Do not clone. Use 4.1 Route B to attach it to GitHub.
Commits show the wrong author	`user.email` not your GitHub address	Redo 1.3
Build stops: `content changed without a version bump`	The working case, caught	Bump the version in `src/<addon>/addon.xml`, rebuild
`check_deps` reports an unresolvable import	A dependency nobody publishes	Move the add-on to `external` and document the vendor repo
Mirror sync: `no published release, falling back to branch head`	Upstream publishes no releases	Normal. Set `"track": "branch"` to make it explicit.
Mirror sync: add-on not found in the tarball	`id` does not match upstream's `addon.xml`	Correct the `id`
Mirror sync fails after an upstream rename	Default branch changed	Set `"ref"` explicitly
Actions run but the commit step fails `403`	Workflow permissions set to read-only	Do 4.5
Actions never run at all	Workflows disabled (usual on a fork)	Actions tab → enable them
Kodi: "Could not connect to repository"	`github_user`/`github_repo` wrong, or repo private	Re-run 4.4; confirm the repo is public
Kodi: repository installs but lists nothing	Index empty, or checksum stale	Re-run 2.3; never hand-edit `docs/`
Kodi: "Failed to install add-on"	Zip name does not match `id-version.zip`	Re-run 2.3; the builder names them
Kodi: "Dependency not met"	Unresolvable `<import>`	Run 2.4
An add-on never updates on one device	Per-add-on auto-update set to Never	Add-on info → Auto-update → On

Symptom	Cause	Fix
Updates appear only after a restart	Normal 24-hour poll cycle	Force a check on the device, or wait

Appendix D — The browser-only route

Everything in Parts 2 to 7 can be done in a web browser, without a terminal, because **the build runs on GitHub rather than on your machine.** You edit a file, GitHub rebuilds the repository and commits the result, and your devices pick it up. This appendix is the parallel path.

D.1 Getting the files onto GitHub

This is the one step where the browser is clumsy — 49 files, 27 MB. Two ways:

Route A — seed once, then never touch a terminal again. Have someone run the single `git push` from Part 3, or do it yourself once. Everything from D.2 onward is pure browser. This is the recommended route.

Route B — no terminal at all.

1. Go to **github.com/new**. The *Create a new repository* page has two sections; set every field as below, then click **Create repository**.

Section	Field	Set it to	Why
1 General	**Owner**	Your account or org, e.g. `StrainBrainOfficial`	This becomes `github_user` in `repo.config.json` and appears in every URL your devices fetch
1 General	**Repository name**	Whatever you like, e.g. `WatchaMe`	Must match `github_repo` in `repo.config.json` **exactly**, including capitals
1 General	**Description**	Optional, anything	Cosmetic; shown on the repo page only
2 Configuration	**Choose visibility**	**Public**	Kodi fetches over plain HTTPS and cannot authenticate. A private repo is unreachable by your devices.
2 Configuration	**Add README**	**Off**	
2 Configuration	**Add .gitignore**	**No .gitignore**	
2 Configuration	**Add license**	**No license**	

> **Why all three must be off.** Any one of them creates an initial commit, so the repository is no longer empty. The **uploading an existing file** shortcut in step 2 disappears, and a later `git push` is rejected until you merge or force. Leave them off and the repository starts clean.

1. On the empty repository page, click **uploading an existing file**.
2. Drag the whole project folder onto the page. Wait for all 49 files to list.
3. Type a commit message and click **Commit changes**.
4. **Check the `.github/workflows/` folder arrived.** Hidden folders sometimes do not survive a drag from Finder or Explorer. If it is missing: **Add file** → **Create new file**, type the path `.github/workflows/sync.yml` — typing slashes creates the folders — paste the workflow contents, and commit.

Nothing in the repository exceeds GitHub's 25 MB web-upload limit.

D.2 Set the workflow permission

Settings → **Actions** → **General** → scroll to **Workflow permissions** → **Read and write permissions** → **Save**.

Without this the build runs and then fails at the final commit with a `403`. It is the same as step 4.5.

D.3 Set your identity

1. Click `repo.config.json` in the file list.
2. Click the **pencil** icon (*Edit this file*).
3. Set `github_user`, `github_repo`, `repo_name` and `provider` as in step 2.2.
4. Scroll down, click **Commit changes**.

The commit triggers a build automatically, because the workflow watches `repo.config.json`.

D.4 Run or watch the build

The workflow runs on any commit touching `addons.json`, `repo.config.json`, `src/` or `scripts/`, and daily at 05:15 UTC. To run it by hand:

Actions tab → **Sync repository** in the left sidebar → **Run workflow** button → **Run workflow**.

Check: the run appears with a green tick. Click it to read the log — it is the same output as the terminal commands in 2.3 and 2.4.

A red cross reading `content changed without a version bump` means D.6 was done without D.7.

D.5 Verify what Kodi will see

Open these three in a browser tab, substituting your values:

```
https://raw.githubusercontent.com/YOURNAME/YOURREPO/main/docs/addons.xml
https://raw.githubusercontent.com/YOURNAME/YOURREPO/main/docs/addons.xml.md5
https://raw.githubusercontent.com/YOURNAME/YOURREPO/main/docs/repository.watchame.zip
```

Check: the first shows XML listing your add-ons; the second shows a single 32-character hash; the third downloads a file. A **404** on any means the values in D.3 do not match the repository's actual address.

The third URL is also the one you enter in Book Two, step 17 — minus the filename.

D.6 Add or remove an add-on

1. Click `addons.template.json` and copy the block you need — Part 3's *Writing entries by hand* section explains which, and D.8 covers deciding `official` versus `mirror` without a terminal.
2. Click `addons.json` → **pencil** → paste your entry into the matching section → **Commit changes**.

D.7 Bump the Toolbox version — the step that matters

1. Navigate to `src` → `script.watchame.toolbox` → `addon.xml`.
2. Click the **pencil**.
3. Increase the version, e.g. `version="1.2.1"` → `version="1.2.2"`.
4. **Commit changes.**

Do D.6 and D.7 as two commits or one, but never D.6 alone: the curated list ships inside the Toolbox, so without a version bump no device ever fetches your change. The build will stop with a red cross if you forget.

D.8 Deciding `official` versus `mirror` without a terminal

Step 3.2 runs the planner, which queries Kodi's official index for you. Without a terminal you cannot run it, so check on the device instead:

Add-ons → **Install from repository** → **Kodi Add-on repository** → browse the category and look for the add-on by name.

Found there, it goes in `official`. Not found, it goes in `mirror`.

D.9 What you give up

Only local testing. Running the build on your own machine catches a mistake in seconds; via the browser you find out when the Actions run goes red a minute later. The checks are identical either way — `build_repo.py` and `check_deps.py` run in both places, so nothing broken reaches a device.

BOOK TWO

Provisioning a Fire TV Stick, step by step

One continuous procedure, steps 1 to 34, taking a Fire TV Stick from factory state to a finished, self-updating Kodi box. Target hardware: **Fire TV Stick 4K (2018, model E9L29Y)** — 1.5 GB RAM, 8 GB storage, Android 7.1+, Kodi 21.1. Notes for the 1 GB gen 1/2 sticks appear where they differ.

Lines marked **Check** tell you what success looks like. Roughly 30 minutes end to end:

Stage	Steps	Time
A Prepare the stick	1–5	3 min
B Install Kodi	6–8	5 min
C Surfshark	9–13	5 min
D Repository	14–20	5 min
E Add-ons	21–23	5 min
F Configure	24–30	5 min
G Verify	31–34	3 min

Doing Stage A step 3 (ADB) costs a minute and saves more than that later — steps 17, 32 and 33 all use it.

> **Do Book One first, all four parts.** This book installs whatever your repository currently carries, so the add-on list has to be settled before you touch the stick — that is Book One, Part 3. Coming here early means provisioning the device twice.

Before you start, have these ready:

- The install URL from Book One, step 4.6
- Your Surfshark login
- The Kodi `armeabi-v7a` APK for your version, or the Downloader app
- Optional but recommended: ADB on your computer, for the verification steps

Stage A — Prepare the stick (steps 1–5)

Where: the TV, with the Fire TV remote. Step 5 is your computer's terminal.

1. On the stick: Settings → My Fire TV → About → click **Fire TV Stick** seven times to reveal Developer Options.

2. Settings → My Fire TV → Developer Options → **Apps from Unknown Sources** → **On**.

3. *(Optional, for the verification steps)* Same menu → **ADB debugging** → **On**.

4. Settings → My Fire TV → About → Network → note the **IP address**.

5. *(Optional)* Pair your computer with the stick. Substitute the address from step 4:

```
adb connect 192.168.1.50:5555
```

Watch the TV: a dialog asks **Allow USB debugging?** — tick *Always allow from this computer* and choose **OK**. Then confirm the pairing took:

```
adb devices
```

Check: a line ending in `device`, e.g. `192.168.1.50:5555 device`.

- An empty list, or `adb: no devices/emulators found` from any later command, means the pairing has not happened. Re-do steps 3 and 4, then this one.
- `unauthorized` instead of `device` means the TV dialog was not accepted. Run `adb disconnect`, connect again, and watch the screen.
- `failed to connect` means the address is wrong, the stick is asleep, or it is on a different network from your computer. Wake it and re-check step 4.

Stage B — Install Kodi (steps 6–8)

Where: the TV, or your computer's terminal for route B.

6. Install Kodi. Pick one route.

Route A — Downloader app on the stick. Open Downloader, type this exact URL, press Go, then Install when it finishes:

```
mirrors.kodi.tv/releases/android/arm/kodi-21.1-Omega-armeabi-v7a.apk
```

Route B — ADB from your computer. Download the same file, then:

```
adb install -r kodi-21.1-Omega-armeabi-v7a.apk
```

> **Two things to get right.** Use **armeabi-v7a**, never arm64-v8a — the stick is 32-bit ARM and the wrong file simply fails to install. And if Kodi is already on the stick, install the **same version it already runs**, or its profile may not survive the change. Newer builds live in the same directory (`21.3` is current at time of writing); browse `mirrors.kodi.tv/releases/android/arm/` to pick one.

7. Launch Kodi once, let it reach the home screen, then quit it. This creates the profile directory.

8. Skip Kodi's setup wizard if offered. Leave the skin as Estuary.

Stage C — Set up Surfshark (steps 9–13)

Where: the TV. Step 13 is your computer's terminal.

Do this **before** any sign-in inside Kodi. Sign-ins bind to the address that completed them, so doing this later means redoing them.

9. On the stick: Amazon appstore → search **Surfshark** → install.

10. Open Surfshark, sign in, connect to a location.

11. In Surfshark's settings, enable **auto-connect on startup**, so a reboot cannot silently drop protection.

12. *(Optional)* Set **Bypasser** if you want specific apps outside the tunnel.

13. Verify the tunnel at OS level — the app's own UI is not proof. *(Needs ADB from step 5.)*

```
adb shell ip addr show tun0
```

Check: a `tun0` interface exists with an IP. If it does not, the stick is not tunnelling no matter what the app displays.

> **Do not use the VPN Manager add-on on this device.** It raises an OpenVPN tunnel from inside Kodi, which an ordinary Android app cannot do on Fire OS. It also bundles profiles for NordVPN, ExpressVPN and PIA only — not Surfshark. It stays in the repository, unticked, for Linux and LibreELEC boxes; Appendix C covers using it with Surfshark there.

Stage D — Install the repository (steps 14–20)

Where: the TV, inside Kodi. Step 17 is far quicker from your terminal.

14. Kodi → Settings (gear) → System → Add-ons → **Unknown sources** → **On**. Accept the warning.

15. Settings → Add-ons → Updates → **Install updates automatically**.

> Step 15 goes before any add-on exists so every one inherits it. This single setting is what makes the stick maintenance-free.

16. Settings → File manager → **Add source** → `<None>`.

17. Enter the install URL from Book One, step 4.6:

```
https://raw.githubusercontent.com/YOURNAME/YOURREPO/main/docs
```

> **Do not type this on the remote.** It is the slowest minute of the whole procedure and a single wrong character fails silently at step 19. With ADB connected (step 5), tap the text field once so the cursor is in it, then type it from your computer:
>
> `adb shell input text "https://raw.githubusercontent.com/YOURNAME/YOURREPO/main/docs"`
>
> The text appears in the field. Check it on screen before pressing OK.

18. Name it `watchame` → OK.

19. Kodi → Add-ons → **Install from zip file** → `watchame` → `repository.watchame.zip`.

Check: a notification confirms the repository add-on installed.

20. Add-ons → **Install from repository** → WatchaMe → Program add-ons → **WatchaMe Toolbox** → Install.

Check: the Toolbox appears under Add-ons → Program add-ons.

> If step 19 fails, the URL in step 17 is wrong. Re-run Book One step 4.4 from your computer.

Stage E — Install the add-ons (steps 21–23)

Where: the TV, inside Kodi.

21. Open **WatchaMe Toolbox** → **Install curated add-ons**.

22. In the picker, untick anything you do not want. For this stick:

Untick	Reason
`skin.copacetic`	40–150 MB and slower menus; Estuary is lighter
`script.plexmod`	Only useful with a Plex server
`pvr.iptvsimple`	Only useful with an IPTV M3U subscription
`service.vpn.manager`	Wrong route on Fire OS — Stage C already covered it
`script.service.janitor`	Only if the stick stores local recordings

Leave ticked: InputStream Adaptive and FFmpeg Direct, a4kSubtitles, Black Bars Never, Up Next, Logfile Uploader, YouTube.

23. Confirm and let it run.

Check: the Toolbox reports what installed. Anything already present is skipped automatically.

Stage F — Configure (steps 24–30)

Where: the TV, inside Kodi.

24. Toolbox → **Apply streaming cache tuning** → select **Fire TV Stick** → Write → **Restart** when prompted.

> Do not pick *Balanced* here. Its 64 MB buffer becomes roughly 192 MB resident, which does not fit alongside Fire OS on 1.5 GB. On a 1 GB gen 1/2 stick, pick *Low memory* instead.

25. Settings → Player → Language → **Default TV show service** and **Default movie service** → set both to **a4kSubtitles**.

> Subtitles do nothing until this is set. It is the single most common "it's broken" report.

26. Settings → Player → Videos → **Allow hardware acceleration – MediaCodec (Surface)** → **On**.

27. Same screen → **Allow hardware acceleration – MediaCodec** → **Off** (redundant with Surface, costs memory).

28. Same screen → **Adjust display refresh rate** → **On start/stop**; **Sync playback to display** → **Off**; **Enable HQ scalers** → **0%**.

29. Settings → Interface → **Show RSS news feeds** → **Off**. Settings → Media → General → **Show media flags** → **Off**.

30. Settings → Services → Control → **Allow remote control via HTTP** → **On**, port 8080.

> Step 30 is what makes remote diagnosis possible later without touching the device.

Stage G — Verify (steps 31–34)

Where: your computer's terminal, except step 34 at the TV.

31. Reboot the stick, then leave Kodi sitting on the home screen for a minute.

32. Check memory and disk from your computer. *(Needs ADB from step 5. Without it, use Toolbox → **Storage report** for the disk figure; there is no on-device view of memory.)*

```
adb shell dumpsys meminfo org.xbmc.kodi | grep 'TOTAL PSS'
adb shell du -sh /sdcard/Android/data/org.xbmc.kodi/files/.kodi
```

Check: against the targets below.

Metric	Stick 4K	1 GB sticks
TOTAL PSS idle	under 600 MB	under 420 MB
`.kodi/` on disk	under 500 MB	under 350 MB
Cold start	under 18 s	under 25 s
Background services	6 or fewer	4 or fewer

33. Confirm auto-update really is set:

```
adb shell "grep addonupdates \
  /sdcard/Android/data/org.xbmc.kodi/files/.kodi/userdata/guisettings.xml"
```

Check: the value is `0`. If it is `1` or `2`, redo step 15.

34. Play something for a minute and confirm it is smooth — no stutter, no audio drifting out of sync. With a USB or Bluetooth keyboard attached, `Ctrl`+`Shift`+`O` overlays the codec panel with a dropped-frame counter, which is the precise version of the same check.

Done. The stick now updates itself. Every add-on tracks its own upstream through the repository, with no further action.

After setup — routine maintenance

When	Do
Monthly	Toolbox → **Storage report**
When storage looks bad	Toolbox → **Clean up caches**
Occasionally	Toolbox → **Clean video/music library**
After adding an add-on	Re-run step 32 to catch a heavy background service

Artwork re-downloads as you browse after a cache clean. That is expected; the first few minutes feel slower.

Troubleshooting

Symptom	Fix
`adb: no devices/emulators found`	Not paired yet — do steps 3, 4 and 5 before any other adb command
`adb` device shows `unauthorized`	The *Allow USB debugging* dialog on the TV was not accepted
Kodi APK will not install	Wrong architecture — use `armeabi-v7a`
Step 19 fails	URL in step 17 is wrong; re-run Book One step 4.4
Repository installs but lists nothing	Book One, Appendix C
Subtitles never appear	Step 25 not done
Buffering on 1080p or 4K	Redo step 24 and pick **Fire TV Stick**
Stutter on high-bitrate video	Step 26 — MediaCodec (Surface) is off
Black screen with audio	Step 27 — disable the non-Surface MediaCodec option
Very slow menus	A third-party skin got installed; switch back to Estuary
Out of space within weeks	Toolbox → Clean up caches; confirm step 24 was applied
Add-ons never update	Step 33 returns something other than `0`
Surfshark shows connected but traffic is not tunnelled	Step 13 shows no `tun0`; reconnect in the app
Everything is slow on a gen 1/2 stick	1 GHz dual-core is the limit — cut the add-on count, expect 1080p only

Appendix A — The hardware

	Stick gen 1 (2014)	Stick gen 2 (2016)	Stick 4K (2018)	Stick Lite / 3rd gen
Model	W87CUN	LY73PR	**E9L29Y**	—
RAM	1 GB	1 GB	**1.5 GB**	1 GB
Storage	8 GB (~5 usable)	8 GB (~5 usable)	**8 GB (~5 usable)**	8 GB
CPU	Dual-core 1.0 GHz	Quad-core 1.3 GHz	**Quad-core 1.7 GHz**	Quad-core 1.7 GHz
Android base	5.1	5.1	**7.1+**	9
Realistic Kodi ceiling	19–20	20	**21+**	21+

Fire OS reserves 350–450 MB. Kodi with Estuary is 250–300 MB. That leaves roughly 250 MB of headroom on a 1 GB stick and 700 MB on the 4K.

Storage, not RAM, is the binding constraint on the 4K. The extra 512 MB buys room for services, but the flash is the same 8 GB and the texture cache grows without limit until something caps it — which is what step 24 does.

On gen 1/2 sticks (Android 5.1) the practical Kodi ceiling is 20; check kodi.tv/download for the current `armeabi-v7a` minimum, as it rises with each release.

Appendix B — What step 24 writes

Toolbox → cache tuning → Fire TV Stick produces `userdata/advancedsettings.xml`, backing up any existing file with a timestamp first:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<advancedsettings>
  <cache>
    <buffermode>1</buffermode>
    <memorysize>41943040</memorysize>
    <readfactor>4</readfactor>
  </cache>
  <gui>
    <algorithmdirtyregions>3</algorithmdirtyregions>
  </gui>
  <imageres>540</imageres>
  <fanartres>720</fanartres>
  <network>
    <curlclienttimeout>20</curlclienttimeout>
    <curllowspeedtime>15</curllowspeedtime>
  </network>
</advancedsettings>
```

- `memorysize` **40 MB.** Kodi allocates roughly three times the configured value, so this is ~120 MB resident. *Low memory* (20 MB → ~60 MB) suits 1 GB sticks; *Balanced* (64 MB → ~192 MB) needs a box with real RAM.
- `imageres` **/** `fanartres`**.** Dropping fanart from 1080 to 720 more than halves texture cache growth, and on a TV that upscales anyway the difference is invisible. This is the most effective single setting against the 8 GB flash filling.
- `algorithmdirtyregions` **3.** Redraw only what changed.
- **Network timeouts.** A dead source fails instead of hanging the UI.

Cache settings take effect only after the restart in step 24.

Appendix C — VPN Manager with Surfshark, on a non-Android box

Only relevant on Linux, LibreELEC or Windows, where an add-on can execute OpenVPN.

1. Toolbox → Install curated add-ons → tick **VPN Manager for OpenVPN**.
2. VPN Manager → settings → provider → **User Defined**.
3. Download Surfshark's `.ovpn` config files from their manual-setup page.
4. Import them with VPN Manager's import wizard.
5. Authenticate with Surfshark's **manual-setup service credentials** from your account dashboard — a separate username and password, **not** your Surfshark login.

Step 5 is the usual failure: the account email and password do not work for manual OpenVPN.

Appendix D — Why there is no snapshot build

The original goal was a "build" — a packaged `userdata` + `addons` snapshot installed by a wizard. That shape is wrong here for four reasons:

1. **It cannot update itself.** Add-ons inside a build are sideloaded, belong to no repository, and never receive updates.
2. **Reinstalling wipes your configuration**, because updating means replacing `userdata`.
3. **Builds rot across Kodi releases** — settings move and the add-on database schema changes.
4. **Builds leak credentials.** `userdata/addon_data/` holds API keys and account logins; packaging it ships your accounts to everyone who installs it.

Stages D–F deliver what people actually want from a build — one action, whole set installed, sensible settings — while every add-on stays individually updatable.

If you genuinely need one (cloning many identical sticks, or no reliable network):

```
cd ~/.kodi
zip -qr ~/mybuild-1.0.0.zip addons userdata \
  -x 'addons/packages/*' 'userdata/Database/*' 'userdata/Thumbnails/*' \
```

```
        '*/__pycache__/*' '*.pyc' '*.log' 'temp/*'
grep -rilE 'token|apikey|password|secret' ~/.kodi/userdata/addon_data/
```

Review whatever that `grep` lists before sharing the zip. Restore by extracting over `.kodi/` with Kodi **force-stopped** — it rewrites `guisettings.xml` on exit and will otherwise overwrite what you restored:

```
adb shell am force-stop org.xbmc.kodi
adb push mybuild-1.0.0.zip /sdcard/Download/
adb shell "cd /sdcard/Android/data/org.xbmc.kodi/files/.kodi && \
  unzip -o /sdcard/Download/mybuild-1.0.0.zip"
```

Install the repository afterwards so the cloned devices resume receiving updates.

BOOK THREE

Worksheet

Worksheet

Fill this in as you follow the guide. Every blank is used by a numbered step, and the step is named beside it.

1. Your values

Fill these in once, in Book One, Part 1.

What	Used by	Your value
GitHub username or org	2.2 `github_user`	
Repository name	2.2 `github_repo`	
Repository display name	2.2 `repo_name`	
Provider / author name	2.2 `provider`	
Install URL	4.6, and Book Two step 17	`https://raw.githubusercontent.com/____/_____/main/docs`
Fire TV Stick IP address	Book Two steps 4, 5	
Kodi version on the stick	Book Two step 6	

2. Add-ons you want

You can fill this in on paper, or skip it and type straight into `addons.wishlist.txt` in the repository root, which the planner in step 3.2 reads directly. Same information either way.

Display name	Add-on ID	GitHub `owner/repo` (if any)	`optional`?	Added
			yes / no	[ ]
				[ ]
				[ ]
				[ ]
				[ ]
				[ ]
				[ ]
				[ ]

Reminder: after adding entries, **bump the Toolbox version** (step 3.4) or no device will ever see them.

3. Book One checklist

Part 1 — GitHub setup *(once per machine; skip if* `gh auth status` *already shows you logged in)*

[ ] 1.1 GitHub account exists

[ ] 1.2 `git`, `gh` and `python3` installed, all reporting a version

[ ] 1.3 `git config --global user.name` and `user.email` set

[ ] 1.4 `gh auth login` done; `gh auth status` shows *Logged in*

[ ] 1.5 Repository directory on this machine

Part 2 — Configure the repository

[ ] 2.1 In the repository directory

[ ] 2.2 Four identity fields set in `repo.config.json`

[ ] 2.3 `build_repo.py` ran; served-from URL matches your values

[ ] 2.4 `check_deps.py` says *all dependencies resolve*

[ ] 2.5 Committed

Part 3 — Choose your add-ons *(before publishing)*

[ ] 3.1 `addons.wishlist.txt` filled in

[ ] 3.2 Planner run — no `!!` lines left

[ ] 3.3 GitHub slugs added for anything flagged

[ ] 3.4 `--apply` run (writes entries, bumps the Toolbox)

[ ] 3.5 Every `why`, `name` and `category` corrected

[ ] 3.6 Build and dependency check both clean

[ ] 3.7 Committed

[ ] 3.8 Add-ons confirmed in `docs/addons.xml`

Part 4 — Publish

[ ] 4.1 Repository created (or remote added), public

[ ] 4.2 Pushed, with no authentication prompt

[ ] 4.3 Actions run green

[ ] 4.4 All three URLs verified (200 / 32-char hash / 200)

[ ] 4.5 Workflow permission confirmed as `write`

[ ] 4.6 Install URL written into section 1 above

4. Book Two checklist *(only after Book One Parts 1–4)*

[ ] **Stage A** (1–5) Developer options on, unknown sources on, IP noted

[ ] **Stage B** (6–8) Kodi installed (`armeabi-v7a`), launched once, quit

[ ] **Stage C** (9–13) Surfshark installed, auto-connect on, `tun0` confirmed

[ ] **Stage D** (14–20) Auto-update set *before* add-ons, source added, repository and Toolbox installed

[ ] **Stage E** (21–23) Curated add-ons installed, unwanted entries unticked

[ ] **Stage F** (24–30) Cache profile applied, subtitle default set, MediaCodec Surface on

[ ] **Stage G** (31–34) Rebooted, memory and disk within targets, `addonupdates` = `0`, playback clean

5. Measured results

From Book Two, step 32. Compare against the targets in the same step.

Metric	Target (Stick 4K)	Measured
TOTAL PSS idle	under 600 MB	
`.kodi/` on disk	under 500 MB	
Cold start	under 18 s	
Background services	6 or fewer	
`general.addonupdates`	`0`	